

Excercise: Power and design analysis

Martin Johnsson

The power of a t-test

In the lecture, we used this code to look at the power of a t-test.

First, we define som parameters, including the standardised effect size (Cohen's d).

```
library(broom)
library(purrr)

a <- 1840
b <- 100
sd <- 191

d <- b / sd
d
```

```
[1] 0.5235602
```

Here is a function to simulate a t-test between two groups, and return the results in a nice data frame format (with `broom::tidy`).

```
simulate_t <- function(a, b, sd, n_group) {
  group <- rep(0:1, each = n_group)
  trait <- a + group * b + rnorm(n_group * 2, 0, sd)
  tidy(t.test(trait ~ group))
}
```

We run this simulation 1000 times (with `purrr::map_dfr`; you could also use a loop).

```
sim_test <- map_dfr(1:1000, function(x) simulate_t(a, b, sd, 60))
sim_test
```

```
# A tibble: 1,000 x 10
  estimate estimate1 estimate2 statistic  p.value parameter conf.low conf.high
  <dbl>      <dbl>      <dbl>      <dbl>    <dbl>      <dbl>      <dbl>      <dbl>
1   -107.      1829.      1935.      -3.25 0.00153        116.      -172.      -41.7
2    -81.1     1828.      1909.      -2.28 0.0243         117.      -152.      -10.7
3   -101.      1829.      1930.      -3.29 0.00131        118.      -162.      -40.3
4    -72.4     1888.      1961.      -2.01 0.0467         117.      -144.       -1.07
5    -76.4     1865.      1941.      -2.27 0.0249         116.      -143.      -9.79
6    -83.4     1840.      1924.      -2.51 0.0134         116.      -149.      -17.7
7   -140.      1832.      1972.      -3.91 0.000155        116.      -211.     -69.1
8    -83.1     1880.      1963.      -2.19 0.0304         118.      -158.       -7.99
9    -91.0     1831.      1922.      -2.40 0.0180         118.      -166.     -15.9
10   -135.      1814.      1949.      -3.83 0.000213        113.      -205.     -65.2
# i 990 more rows
# i 2 more variables: method <chr>, alternative <chr>
```

This gives us a data frame of output from simulation replicates, from which we can calculate any summaries we want.

We estimate the power, Type S error rate and Type M exaggeration ratio at a threshold of $p < 0.05$.

```
est_power <- sum(sim_test$p.value < 0.05)/nrow(sim_test)
est_S_error <- sum(sim_test$p.value < 0.05 & sim_test$estimate > 0)/nrow(sim_test)
est_M_error <- mean(abs(sim_test$estimate[sim_test$p.value < 0.05])/b)

est_power
```

```
[1] 0.813
```

```
est_S_error
```

```
[1] 0
```

```
est_M_error
```

```
[1] 1.102802
```

We can compare that to the power of a t-test from formulas, implemented in R.

```
power.t.test(delta = b, sd = sd, sig.level = 0.05, power = 0.8)
```

Two-sample t test power calculation

```
      n = 58.24311
delta = 100
      sd = 191
sig.level = 0.05
      power = 0.8
alternative = two.sided
```

NOTE: n is number in *each* group

Exercises

Choose the exercises that interest you.

- **Effect size and sample size.** Make a grid of reasonable effect sizes (expressed as Cohen's d, grams or however you prefer) and sample sizes, and plot the resulting power from simulations and from calculations. How does the power vary with sample size and effect size?
- **Type M and Type S error.** Calculate Type M and S error on the same grid of parameters. When is the risk of type S error high? When is the exaggeration ratio high?
- **Type 1 error rate.** If we simulate two groups with no real difference between them, we can quantify the type 1 error rate. Create a simulation without effect, and estimate the type 1 error rate. Plot the type 1 error rate for different significance thresholds to check that the t-test behaves as you expect. Now change something about the error model – you can make the variance bigger in one of the groups, change the distribution of the error to a t-distribution or another fat-tailed distribution, etc – and see how this affects type 1 error rate.
- **Multiple testing.** Expand the simulation to generate a large number of independent variables and apply a t-test to each of them. There are multiple methods to account for such multiple testing, some of them implemented in R's `p.adjust` function. Choose a couple of potential correction methods and check their effect on simultaneous type 1 error rate and power.
- **A model of your own.** Think of an example of a statistical analysis that you have performed or will perform in your own research. Ideally it should be relatively small, so perhaps you need to simplify it somewhat, take away extra predictors and so on. Write

simulation code, using functions such as `rnorm`, `sample` etc to generate fake data from with and without the effect that you are interested in measuring. Use this simulation to quantify power and error rates.